

Unidad Didáctica

**Datos Estructurados,
Unidad V**

Contenido

Introducción	3
Desarrollo de Contenidos	4
Tipos de Datos Estructurados: Arreglo.....	4
Introducción.....	4
Arreglos unidimensionales.....	5
Asignación, lectura y escritura de valores a un Arreglo.....	7
Arreglos multidimensionales	8
Representación de los Arreglos en memoria	9
Operaciones con Arreglos multidimensionales.....	11
Arreglos como parámetros a módulos	13
Ordenación y búsqueda de Arreglos.....	15
Tipos de Datos Estructurados: Registro	17
Definición de registros	17
Inicialización de registros	18
Operaciones con registros	20
Registros a funciones.....	22
Bibliografía y Webgrafía	24
Glosario	24

Introducción



¡Bienvenido! En el mundo de la programación, los tipos de datos estructurados son fundamentales para el desarrollo de aplicaciones. En este caso, hablaremos de dos tipos específicos: arreglos y registros.

Comencemos hablando de los arreglos. Estos son una colección de datos del mismo tipo que se almacenan consecutivamente en la memoria. Los arreglos pueden ser unidimensionales o multidimensionales, lo que significa que pueden contener una o varias dimensiones. Para manejar los arreglos, se utilizan operaciones como la asignación, la lectura y la escritura de valores. Además, se pueden usar arreglos como parámetros en módulos y también se pueden ordenar y buscar.

Por otro lado, los registros son una estructura de datos que contienen un conjunto de valores con nombres relacionados. Cada uno de estos valores se conoce como campo. Los registros pueden ser inicializados y se pueden realizar operaciones con ellos, como la asignación de valores a los campos y la lectura de los valores de los campos. También se pueden utilizar registros como argumentos en funciones.

En este tema, profundizaremos en los arreglos y los registros, analizando su estructura, las operaciones que se pueden realizar con ellos, cómo se representan en la memoria y cómo se utilizan como parámetros. También veremos cómo se pueden ordenar y buscar arreglos, y cómo se pueden inicializar registros. ¡Empecemos a aprender sobre estos tipos de datos estructurados!

Tipos de Datos Estructurados: Arreglo

Introducción

Los arreglos son un tipo de datos estructurados que se utilizan en programación para almacenar una colección de elementos del mismo tipo. Se pueden pensar como una lista ordenada de valores, donde cada valor está identificado por un índice. Los arreglos son muy útiles en programación porque permiten acceder y manipular conjuntos de datos de manera eficiente.

En Python, los arreglos se pueden crear utilizando la librería **array**. Para crear un arreglo, primero se debe importar la librería y luego se puede utilizar la función **array()** para crear el arreglo. Por ejemplo, el siguiente código crea un arreglo de enteros:

```
import array as arr

mi_arreglo = arr.array('i', [1, 2, 3, 4, 5])
```

En este caso, la letra 'i' se utiliza para indicar que los elementos del arreglo son enteros. Otras letras se pueden utilizar para indicar otros tipos de datos, como 'f' para números de punto flotante o 'b' para bytes.

Una vez que se ha creado un arreglo, se pueden acceder y manipular los elementos utilizando sus índices. Por ejemplo, para acceder al segundo elemento del arreglo, se puede utilizar el siguiente código:

```
print(mi_arreglo[1])
```

Lo cual imprimirá el valor **2**.

Además de acceder a elementos individuales, también es posible realizar operaciones en todo el arreglo, como sumar todos sus elementos o encontrar el valor máximo. Por ejemplo, para sumar todos los elementos de un arreglo, se puede utilizar la función **sum()** de Python:

```
total = sum(mi_arreglo)
```

En conclusión, los arreglos son un tipo de datos estructurados útiles en programación para almacenar y manipular conjuntos de datos de manera eficiente. En Python, se pueden crear arreglos utilizando la librería **array** y acceder a sus elementos utilizando sus índices.

Arreglos unidimensionales

Los arreglos unidimensionales son una estructura de datos común en programación que se utiliza para almacenar una colección de elementos de un solo tipo. Los arreglos unidimensionales se representan como una secuencia de elementos, donde cada elemento se identifica por su posición dentro del arreglo. Estos arreglos son útiles porque permiten acceder y manipular conjuntos de datos de manera eficiente. En Python, los arreglos unidimensionales se pueden crear utilizando la librería **array**. Para crear un arreglo unidimensional, primero se debe importar la librería y luego se puede utilizar la función **array()** para crear el arreglo. Por ejemplo, el siguiente código crea un arreglo unidimensional de enteros:

```
import array as arr
```

```
mi_arreglo = arr.array('i', [1, 2, 3, 4, 5])
```

En este caso, la letra 'i' se utiliza para indicar que los elementos del arreglo son enteros. Otras letras se pueden utilizar para indicar otros tipos de datos, como 'f' para números de punto flotante o 'b' para bytes.

Una vez que se ha creado un arreglo unidimensional, se pueden acceder y manipular los elementos utilizando sus índices. Por ejemplo, para acceder al segundo elemento del arreglo, se puede utilizar el siguiente código:

```
print(mi_arreglo[1])
```

Lo cual imprimirá el valor **2**.

Además de acceder a elementos individuales, también es posible realizar operaciones en todo el arreglo, como sumar todos sus elementos o encontrar el valor máximo. Por ejemplo, para sumar todos los elementos de un arreglo unidimensional, se puede utilizar la función **sum()** de Python:

```
total = sum(mi_arreglo)
```

En conclusión, los arreglos unidimensionales son una estructura de datos común en programación que se utiliza para almacenar y manipular conjuntos de datos de manera eficiente. En Python, se pueden crear arreglos unidimensionales utilizando la librería **array** y acceder a sus elementos utilizando sus índices.

Asignación, lectura y escritura de valores a un Arreglo

En programación, los arreglos son una estructura de datos común que se utiliza para almacenar y manipular conjuntos de elementos del mismo tipo. Una vez que se ha creado un arreglo, se pueden asignar, leer y escribir valores en sus elementos utilizando índices.

Para asignar un valor a un elemento en un arreglo, se puede utilizar el siguiente código en Python:

```
mi_arreglo = [1, 2, 3, 4, 5]
```

```
mi_arreglo[2] = 10
```

En este ejemplo, se crea un arreglo **mi_arreglo** con los valores **[1, 2, 3, 4, 5]**. Luego, se asigna el valor **10** al tercer elemento del arreglo (que tiene un índice de **2**) utilizando la sintaxis **mi_arreglo[2] = 10**.

Para leer un valor de un elemento en un arreglo, se puede utilizar la sintaxis **mi_arreglo[i]**, donde **i** es el índice del elemento que se desea leer. Por ejemplo, el siguiente código lee el valor del segundo elemento del arreglo **mi_arreglo**:

```
print(mi_arreglo[1])
```

Lo cual imprimirá el valor **2**.

Para escribir valores en todos los elementos de un arreglo, se puede utilizar un ciclo **for**. Por ejemplo, el siguiente código escribe el valor **0** en cada elemento del arreglo **mi_arreglo**:

En este caso, la función **len()** se utiliza para obtener el número de elementos en el arreglo, y la sintaxis **mi_arreglo[i] = 0** se utiliza para escribir el valor **0** en el elemento **i** del arreglo.

En conclusión, asignar, leer y escribir valores en un arreglo es una operación común en programación. En Python, se puede utilizar la sintaxis **mi_arreglo[i]** para acceder a elementos individuales del arreglo y la sintaxis **mi_arreglo[i] = valor** para asignar valores a elementos individuales del arreglo. Además, se puede utilizar un ciclo **for** para escribir valores en todos los elementos del arreglo.

Arreglos multidimensionales

En programación, un arreglo multidimensional es una estructura de datos que permite almacenar y manipular conjuntos de elementos organizados en dos o más dimensiones. Los arreglos multidimensionales son comunes en aplicaciones que manejan datos tabulares, como hojas de cálculo o matrices matemáticas.

En Python, los arreglos multidimensionales se pueden crear utilizando listas anidadas. Por ejemplo, el siguiente código crea un arreglo bidimensional de tamaño 3x3:

```
for i in range(len(mi_arreglo)):
    for j in range(len(mi_arreglo[i])):
        print(mi_arreglo[i][j])
```

En este caso, se utilizan dos ciclos **for** anidados para recorrer todas las filas y columnas del arreglo **mi_arreglo**, y se utiliza la sintaxis **mi_arreglo[i][j]** para acceder a cada elemento individual.

En conclusión, los arreglos multidimensionales son una estructura de datos importante en programación que permiten almacenar y manipular datos organizados en dos o más dimensiones. En Python, los arreglos multidimensionales se pueden crear utilizando listas anidadas, y se pueden acceder a sus elementos utilizando la sintaxis **mi_arreglo[i][j]**. Además, se pueden utilizar ciclos **for** anidados para recorrer todos los elementos de un arreglo multidimensional.

Representación de los Arreglos en memoria

En programación, la representación en memoria de los arreglos es un tema importante para entender cómo se almacenan y manipulan los datos en un programa. En general, un arreglo se almacena en una región contigua de memoria, donde cada elemento ocupa una posición consecutiva en la memoria.

En Python, los arreglos se representan internamente como listas, que a su vez están formadas por referencias a objetos. Cada elemento del arreglo es un objeto individual en la memoria, y la lista mantiene una referencia a cada uno de estos objetos.

Por ejemplo, considere el siguiente código que crea un arreglo de tamaño 5 y asigna valores a sus elementos:

```
mi_arreglo = [0] * 5
```

```
mi_arreglo[0] = 1
```

```
mi_arreglo[2] = 3
```

```
mi_arreglo[4] = 5
```

En este ejemplo, el arreglo **mi_arreglo** tiene cinco elementos, todos inicializados en cero. Luego, se asignan valores a los elementos en las posiciones 0, 2 y 4. La representación en memoria de este arreglo podría ser la siguiente:

```
|1|0|3|0|5|
```

En este caso, cada elemento del arreglo ocupa cuatro bytes en la memoria, y se almacenan consecutivamente en la región de memoria reservada para el arreglo. La posición de cada elemento en el arreglo se determina por su índice, y la posición en la memoria se calcula multiplicando el índice por el tamaño del elemento y sumando la dirección de inicio del arreglo.

Es importante tener en cuenta que la representación en memoria de los arreglos puede variar dependiendo del lenguaje de programación y la arquitectura del sistema. En algunos casos, por ejemplo, los arreglos pueden estar representados por punteros a memoria en lugar de valores directos. Además, algunos lenguajes de programación permiten la creación de arreglos de tipos de datos complejos, como objetos o estructuras, que pueden requerir una representación en memoria más compleja.

En conclusión, la representación en memoria de los arreglos es un concepto importante en programación que permite entender cómo se almacenan y manipulan los datos en un programa. En Python, los arreglos se representan internamente como listas de referencias a objetos, y cada elemento del arreglo ocupa una posición consecutiva en la memoria. La representación en memoria de los arreglos puede variar dependiendo del lenguaje de programación y la arquitectura del sistema.

Operaciones con Arreglos multidimensionales

Los arreglos multidimensionales son una estructura de datos común en programación, que permiten almacenar y manipular datos en dos o más dimensiones. Las operaciones con arreglos multidimensionales son similares a las operaciones con arreglos unidimensionales, pero se aplican en dos o más dimensiones.

Una de las operaciones más comunes con arreglos multidimensionales es la asignación de valores a los elementos del arreglo. Para esto, se utiliza la notación de corchetes para indicar las posiciones de los elementos. Por ejemplo, si se tiene un arreglo bidimensional llamado **mi_matriz** y se desea asignar el valor **4** a la posición **(1,2)**, se puede hacer de la siguiente manera:

```
mi_matriz[1][2] = 4
```

En este ejemplo, el primer corchete indica la fila y el segundo corchete indica la columna de la posición deseada.

Otra operación común con arreglos multidimensionales es el recorrido o iteración sobre los elementos del arreglo. Para esto, se puede utilizar un bucle anidado para recorrer cada fila y columna del arreglo. Por ejemplo, el siguiente código recorre una matriz de tamaño 3x3 e imprime cada uno de sus elementos:

```
mi_matriz = [[1,2,3], [4,5,6], [7,8,9]]
```

```
for i in range(3):
```

```
for j in range(3):  
  
    print(mi_matriz[i][j])
```

En este ejemplo, se utiliza un bucle **for** anidado para recorrer cada fila y columna del arreglo. La variable **i** indica la fila y la variable **j** indica la columna de la posición actual.

Otras operaciones comunes con arreglos multidimensionales incluyen la suma, la resta, la multiplicación y la transposición. Estas operaciones se realizan de manera similar a como se hacen con los arreglos unidimensionales, pero se aplican en cada dimensión del arreglo.

Por ejemplo, la suma de dos matrices de tamaño 3x3 se puede realizar de la siguiente manera:

```
matriz1 = [[1,2,3], [4,5,6], [7,8,9]]  
  
matriz2 = [[9,8,7], [6,5,4], [3,2,1]]  
  
resultado = [[0,0,0], [0,0,0], [0,0,0]]  
  
for i in range(3):  
  
    for j in range(3):  
  
        resultado[i][j] = matriz1[i][j] + matriz2[i][j]
```

En este ejemplo, se crea una matriz vacía llamada **resultado** y se utiliza un bucle anidado para sumar los elementos de las matrices **matriz1** y **matriz2** y almacenar el resultado en la matriz **resultado**.

En conclusión, las operaciones con arreglos multidimensionales son similares a las operaciones con arreglos unidimensionales, pero se aplican en dos o más dimensiones. Las operaciones comunes incluyen la asignación de valores, el recorrido o iteración sobre los elementos, la suma, la resta, la multiplicación y la transposición. Estas operaciones se realizan de manera similar a como se hacen con los arreglos unidimensionales, pero se aplican en cada dimensión del arreglo.

Arreglos como parámetros a módulos

En Python, los arreglos se pueden pasar como parámetros a los módulos, lo que permite la manipulación de los datos de los arreglos desde diferentes módulos. Al pasar arreglos como parámetros, los módulos pueden realizar operaciones en los datos de los arreglos y devolver los resultados a otros módulos que los llaman.

Para pasar un arreglo como parámetro, simplemente se especifica el nombre del arreglo como uno de los parámetros en la definición de la función del módulo. Luego, se puede acceder a los elementos del arreglo dentro del módulo utilizando el nombre del arreglo y el índice correspondiente.

A continuación, se muestra un ejemplo de cómo pasar un arreglo como parámetro a un módulo en Python:

```
# Definir una función que tome un arreglo como parámetro
```

```
def suma_arreglo(arreglo):
```

```
    suma = 0
```

```
for i in arreglo:
```

```
    suma += i
```

```
return suma
```

```
# Definir un arreglo y llamar a la función con el arreglo como parámetro
```

```
mi_arreglo = [1, 2, 3, 4, 5]
```

```
resultado = suma_arreglo(mi_arreglo)
```

```
# Imprimir el resultado
```

```
print("La suma del arreglo es:", resultado)
```

En este ejemplo, se define una función llamada "suma_arreglo" que toma un arreglo como parámetro. Dentro de la función, se itera a través de los elementos del arreglo y se suma su valor a una variable "suma". Finalmente, la función devuelve la suma de los elementos del arreglo.

Se define un arreglo llamado "mi_arreglo" que contiene los números del 1 al 5. Se llama a la función "suma_arreglo" con "mi_arreglo" como parámetro y se asigna el resultado a la variable "resultado". Luego, se imprime el resultado en la consola.

En resumen, pasar arreglos como parámetros a módulos en Python permite la manipulación de los datos del arreglo en diferentes partes del código, lo que puede facilitar la programación modular y la reutilización de código.

Ordenación y búsqueda de Arreglos

La ordenación y la búsqueda de arreglos son operaciones comunes en la programación que permiten organizar los datos y buscar información específica dentro de una colección de elementos. En Python, existen varias funciones integradas que facilitan estas operaciones para arreglos.

La ordenación de arreglos implica la reorganización de los elementos de un arreglo en un orden específico. La función integrada "sorted()" en Python ordena los elementos de un arreglo en orden ascendente y devuelve una nueva lista ordenada. Por ejemplo:

```
# Ordenar un arreglo en orden ascendente

arr = [5, 2, 8, 3, 1]

arr_ordenado = sorted(arr)

print(arr_ordenado) # [1, 2, 3, 5, 8]
```

La búsqueda de arreglos implica la localización de un elemento específico dentro de un arreglo. La función integrada "index()" en Python devuelve el índice del primer elemento que coincide con un valor específico dentro de un arreglo. Por ejemplo:

```
# Encontrar el índice de un elemento específico en un arreglo

arr = [5, 2, 8, 3, 1]

indice = arr.index(8)

print(indice) # 2
```

Si el valor específico no está presente en el arreglo, la función "index()" genera una excepción "ValueError".

Otra forma de buscar elementos en un arreglo es mediante la búsqueda binaria. La búsqueda binaria es una técnica de búsqueda más eficiente que la búsqueda lineal, ya que divide el arreglo en dos mitades y compara el elemento buscado con el elemento del medio. Si el elemento buscado es menor que el elemento medio, la búsqueda continúa en la mitad inferior del arreglo, y si es mayor, continúa en la mitad superior. Este proceso se repite hasta que se encuentra el elemento buscado o se determina que no está presente en el arreglo.

En Python, la función "bisect()" en el módulo "bisect" implementa la búsqueda binaria en un arreglo ordenado y devuelve el índice donde se encuentra o se debería insertar el elemento buscado. Por ejemplo:

```
# Búsqueda binaria en un arreglo ordenado

import bisect

arr_ordenado = [1, 2, 3, 5, 8]

indice = bisect.bisect_left(arr_ordenado, 5)

print(indice) # 3
```

En este ejemplo, se utiliza la función "bisect_left()" para buscar el valor 5 en el arreglo "arr_ordenado". La función devuelve el índice 3, que es donde se encuentra el valor 5 en el arreglo.

En resumen, la ordenación y la búsqueda de arreglos son operaciones comunes en la programación que se pueden realizar en Python utilizando varias funciones

integradas. La función "sorted()" ordena los elementos de un arreglo, la función "index()" busca un valor específico dentro de un arreglo y la función "bisect()" implementa la búsqueda binaria en un arreglo ordenado. Estas funciones pueden ser útiles en diversas aplicaciones de programación.

Tipos de Datos Estructurados: Registro

Definición de registros

En programación, un registro es una estructura de datos que permite almacenar distintos tipos de información en una sola variable. Es similar a una estructura en otros lenguajes de programación. Cada registro tiene uno o más campos que pueden ser de diferentes tipos de datos, como enteros, flotantes, cadenas de caracteres, etc.

Para definir un registro en Python, se utiliza la palabra clave "class". Dentro de la clase se definen los campos y los métodos que pueden ser utilizados con los registros. Cada campo se define como una variable de la clase, y los métodos son funciones que operan sobre los datos del registro.

A continuación, se muestra un ejemplo de cómo definir un registro en Python:

```
class Persona:

    def __init__(self, nombre, edad):

        self.nombre = nombre

        self.edad = edad

p1 = Persona("Juan", 25)
```

```
print(p1.nombre)
```

```
print(p1.edad)
```

En este ejemplo, se ha definido un registro llamado "Persona" que tiene dos campos: "nombre" y "edad". Se ha creado una instancia de la clase "Persona" llamada "p1" con los valores "Juan" y 25. Después se imprime el valor de los campos "nombre" y "edad" de la instancia "p1".

Los registros pueden ser muy útiles para organizar y manejar grandes cantidades de datos. Al igual que con los arreglos, se pueden utilizar registros como parámetros de funciones y métodos. Además, los registros también pueden ser utilizados para crear estructuras de datos más complejas, como listas enlazadas y árboles.

En resumen, los registros son una estructura de datos que permiten almacenar información de diferentes tipos en una sola variable. Se definen utilizando la palabra clave "class" en Python y pueden contener campos y métodos. Los registros son útiles para manejar grandes cantidades de datos y pueden ser utilizados en la creación de estructuras de datos más complejas.

Inicialización de registros

La inicialización de registros es el proceso de asignar valores iniciales a los campos de un registro en particular. En Python, se pueden inicializar registros mediante el método **__init__()** de la clase.

El método **__init__()** se utiliza para asignar valores iniciales a los campos de un registro cuando se crea una nueva instancia de la clase. El método toma como argumentos los valores iniciales para cada campo del registro.

A continuación, se muestra un ejemplo de cómo inicializar un registro en Python:

```
class Persona:

    def __init__(self, nombre, edad):

        self.nombre = nombre

        self.edad = edad

p1 = Persona("Juan", 25)
```

En este ejemplo, se ha definido un registro llamado "Persona" que tiene dos campos: "nombre" y "edad". En el método **__init__()**, se han asignado valores iniciales para los campos "nombre" y "edad" utilizando los argumentos "nombre" y "edad". Luego, se ha creado una instancia de la clase "Persona" llamada "p1" con los valores "Juan" y 25.

Es importante destacar que la inicialización de registros es una parte fundamental de la programación orientada a objetos, ya que permite crear instancias de la clase con valores específicos para cada campo. Esto hace que los registros sean muy versátiles y adaptables a diferentes situaciones y necesidades.

En resumen, la inicialización de registros es el proceso de asignar valores iniciales a los campos de un registro en particular. En Python, se utiliza el método **__init__()** de la clase para inicializar los registros con valores específicos para cada campo. La inicialización de registros es una parte fundamental de la programación orientada a objetos y permite crear instancias de la clase con valores específicos para cada campo.

Operaciones con registros

Los registros son estructuras de datos en programación que agrupan diferentes tipos de datos bajo un mismo nombre. Cada campo del registro tiene un nombre y un tipo de datos asociado. Los registros permiten trabajar con datos relacionados y organizados de una manera más eficiente y estructurada.

Las operaciones con registros en programación son similares a las operaciones con arreglos, pero en lugar de acceder a los elementos por índices, se accede a los campos del registro por sus nombres.

A continuación, se presentan algunas operaciones comunes que se pueden realizar con registros:

1. Acceder a los campos del registro: Los campos del registro se pueden acceder utilizando el nombre del campo seguido del operador ".". Por ejemplo, si se tiene un registro "Persona" con campos "nombre" y "edad", se puede acceder al campo "nombre" de la siguiente manera:

```
p1 = Persona("Juan", 25)
```

```
print(p1.nombre) # Imprime "Juan"
```

2. Modificar los campos del registro: Los campos del registro se pueden modificar de la misma manera que se accede a ellos. Por ejemplo, si se quiere modificar el campo "edad" del registro "Persona", se puede hacer de la siguiente manera:

```
p1.edad = 26
```

3. Comparar registros: Se pueden comparar dos registros utilizando los operadores de comparación. Los registros se comparan campo por campo, en orden de declaración. Por ejemplo:

```
p1 = Persona("Juan", 25)
```

```
p2 = Persona("Ana", 30)
```

```
if p1 < p2:
```

```
    print("p1 es menor que p2")
```

```
else:
```

```
    print("p1 es mayor o igual que p2")
```

En este ejemplo, se están comparando los registros "p1" y "p2" de la clase "Persona" por el campo "nombre", que se declaró primero en la definición de la clase.

4. Asignar registros a variables: Los registros se pueden asignar a variables de la misma manera que cualquier otro tipo de dato. Por ejemplo:

```
p1 = Persona("Juan", 25)
```

```
p2 = p1
```

En este ejemplo, se está asignando el registro "p1" a la variable "p2". Ambas variables apuntan al mismo registro en la memoria.

En resumen, las operaciones con registros en programación permiten acceder, modificar y comparar campos de registros utilizando sus nombres. También se pueden asignar registros a variables. Los registros son estructuras de datos útiles para agrupar datos relacionados y organizarlos de una manera más eficiente y estructurada.

Registros a funciones

En programación, los registros se utilizan para agrupar diferentes variables de diferentes tipos en una sola unidad lógica. A veces, se requiere pasar esta unidad lógica como un solo parámetro a una función. En estos casos, los registros se pasan a una función como un parámetro. Esta operación se llama pasar un registro a una función.

Para pasar un registro a una función, se puede usar el mismo método que se usa para pasar cualquier otra variable a una función en un lenguaje de programación determinado. La función debe estar definida para recibir un parámetro del tipo del registro que se va a pasar.

A continuación, se muestra un ejemplo en Python para pasar un registro a una función:

```
class Alumno:

    def __init__(self, nombre, edad):

        self.nombre = nombre

        self.edad = edad
```

```
def mostrar_alumno(alumno):  
  
    print("Nombre:", alumno.nombre)  
  
    print("Edad:", alumno.edad)  
  
  
alumno1 = Alumno("Juan", 25)  
  
mostrar_alumno(alumno1)
```

En este ejemplo, se define la clase **Alumno** con dos variables de instancia: **nombre** y **edad**. Luego se define la función **mostrar_alumno** que toma un objeto **Alumno** como parámetro y muestra los valores de sus variables de instancia. Por último, se crea un objeto **Alumno** llamado **alumno1** y se pasa a la función **mostrar_alumno**.

En conclusión, los registros pueden ser pasados a funciones como cualquier otro tipo de variable en un lenguaje de programación. Al utilizar registros en una función, se puede agrupar datos relacionados de una manera lógica y permitir que las funciones trabajen con ellos de manera más efectiva.

Bibliografía y Webgrafía

Piattini, M., García, F. J., Genero, M., & Rodriguez, J. I. (2006). Estructuras de datos en Java: desde la idea hasta la implementación. Pearson Educación.

Farré, C. (2020). Python para todos: Explorando la programación side by side con Python

García, F. (2017). Fundamentos de programación. Alfaomega.

Farré, J. (2019). Python: Curso práctico para científicos e ingenieros. Marcombo.

Glosario

Arreglo: Estructura de datos que almacena una colección de elementos del mismo tipo.

Arreglo multidimensional: Arreglo que tiene más de una dimensión, es decir, que contiene elementos organizados en filas y columnas.

Arreglo unidimensional: Arreglo que tiene una sola dimensión, es decir, que contiene elementos organizados en una sola línea.

Asignación: Operación que consiste en asignar un valor a una variable o a un elemento de un arreglo.

Búsqueda: Operación que consiste en buscar un elemento en un arreglo y determinar si está presente o no.

Definición de registros: Proceso de definir una estructura de datos que contiene varias variables de diferentes tipos.

Inicialización: Operación que consiste en asignar un valor inicial a una variable o a un arreglo.

Lectura: Operación que consiste en obtener el valor de una variable o de un elemento de un arreglo.

Operaciones con arreglos: Diversas operaciones que se pueden realizar con arreglos, como asignación, lectura, escritura, ordenación y búsqueda.

Operaciones con registros: Diversas operaciones que se pueden realizar con registros, como asignación, lectura y escritura de valores.

Ordenación: Operación que consiste en ordenar los elementos de un arreglo de acuerdo a un criterio determinado.

Registro: Estructura de datos que agrupa varias variables de diferentes tipos en una sola unidad lógica.

Representación de arreglos en memoria: Forma en que se almacenan los elementos de un arreglo en la memoria del computador.

Tipos de datos estructurados: Arreglo: Tipo de datos estructurado que almacena una colección de elementos del mismo tipo en una estructura unidimensional.

Tipos de datos estructurados: Registro: Tipo de datos estructurado que agrupa varias variables de diferentes tipos en una sola unidad lógica.

Variables: Identificador utilizado para referirse a una ubicación en la memoria del computador donde se almacena un valor.